contains the business logic, doesn't need to know where the data comes from, as long as it gets access. In chapter 3, we added an implementation of the repository interface to store data in memory. Now that we're building the persistence layer, we'll need a different implementation to access data from PostgreSQL.

The good news is that we can use Spring Data repositories, a technical solution that provides access to data from a data store independently of the specific persistence technology used. It's one of the most valuable features of Spring Data, because we can use the same repository abstraction in any persistence scenario, whether relational or non-relational.

USING A DATA REPOSITORY

When using Spring Data repositories, your responsibility is limited to defining an interface. At startup time, Spring Data will generate an implementation for your interface on the fly. In the Catalog Service project (catalog-service), go ahead and delete the `InMemoryBookRepository` class.

Let's now see how we can refactor the `BookRepository` interface from the Catalog Service project. First, it should extend one of the available `Repository` interfaces provided by Spring Data. Most Spring Data modules add `Repository` implementations specific to the data source supported. The Catalog Service application requires standard CRUD operations on `Book` objects, so you can make the `BookRepository` interface extend from `CrudRepository`.

`CrudRepository` provides methods to perform CRUD operations, including `save()` and `findAll()`, so you can remove their explicit declarations from your interface. The default methods defined by